Step 1: Identify the Smells

Look at the code carefully. Common smells to check first:

- **Long Methods** → Too many lines, nested loops/conditionals
- **Large Classes** → Too many fields and methods
- **Long Parameter List** → Functions with many arguments
- **Duplicate Code** → Same code repeated in multiple places
- **Switch Statements / Conditionals** → Complex logic scattered
- **Temporary Fields** → Fields used only sometimes
- **Feature Envy / Middle Man** → Methods that use other class data more than own

💡 **Tip:** Check **class by class** and **method by method**.

Step 2: Prioritize

Start with **smells that are easy to fix** or cause **most confusion**:

1. Duplicate Code → Extract Method
2. Long Methods → Split into smaller methods
3. Switch / If-Else chains → Use Polymorphism or Strategy
4. Temporary Fields → Make local variables or Method Object
5. Long Parameter List → Introduce Parameter Object

Step 3: Apply Refactorings

- **Extract Method** → for long methods or repeated code

- **Move Method / Field** → for Feature Envy or Inappropriate Intimacy
- **Introduce Parameter Object / Preserve Whole Object** → for long parameter lists
- **Replace Conditional with Polymorphism** → for switch statements
- **Encapsulate Field / Collection** → for data class or primitive obsession
- **Inline Class / Collapse Hierarchy** → for Lazy Class, Dead Code

---

Step 4: Test Frequently

- After each small refactor, **run the code** to ensure it still works.
- Fix one smell at a time → easier to debug

Step 5: Repeat

- Once the obvious smells are gone, look again for **minor smells**:
    - Comments → Rename Methods instead
    - Temporary fields
    - Middle Man → Remove Middle Man

Before practice 1:

```
class Order {
    int tempDiscount;  // temporary field
    List<Item> items;
    String type;  // type code
```

```
class Order {
    List<Item> items;

    void printItems() {
        for(Item i: items) i.printInfo();
    }

    int calculateTotal() {
        int total = 0;
        for(Item i: items) total += i.getPrice();
```

```
    void processOrder(boolean specialDay) { // Long method + temporary field

        if (specialDay) tempDiscount = 10; else tempDiscount = 0;

        System.out.println("Discount: " + tempDiscount);



        // Duplicate code

        for(Item i: items){

            System.out.println("Item: " + i.name);

            System.out.println("Price: " + i.price);

        }

        switch(type) {

            case "Car": System.out.println("Apply car rules"); break;

            case "Bike": System.out.println("Apply bike rules"); break;

            case "Truck": System.out.println("Apply truck rules"); break;    }

        int total = 0;

    for(Item i: items) total += i.price;

 System.out.println("Total: " + (total -
        return total;
    }

    void processOrder(boolean specialDay) {
        int discount = specialDay ? 10 : 0;
        System.out.println("Discount: " + discount);
        printItems();
        System.out.println("Total: " + (calculateTotal() - discount));
    }
}

abstract class VehicleOrder {
    abstract void applyRules();
}

class CarOrder extends VehicleOrder {
    void applyRules() { System.out.println("Apply car rules"); }
}

class BikeOrder extends VehicleOrder {
    void applyRules() { System.out.println("Apply bike rules"); }
}

class TruckOrder extends VehicleOrder {
    void applyRules() { System.out.println("Apply truck rules"); }
}

class Item {
    private String name;
    private int price;

    Item(String name, int price) { this.name = name; this.price = price; }

    void printInfo() {
        System.out.println("Item: " + name);
        System.out.println("Price: " + price);
    }

    int getPrice() { return price; }
```

```
tempDiscount));}}

class Item {

    String name;

    int price;

}
```

Another

```
class OnlineShop {
    int tempDiscount;  // Temporary Field
    List<Product> products;
    String type;       // Type code for category
    String customerType; // Type code for
customer

    void checkout(boolean specialDay, boolean
vipCustomer) { // Long Method
        // Temporary field
        if (specialDay) tempDiscount = 10; else
tempDiscount = 0;
        if (vipCustomer) tempDiscount += 5;
        System.out.println("Discount applied: " +
tempDiscount);

        // Duplicate code
        for(Product p: products){
            System.out.println("Product: " +
p.name);
            System.out.println("Price: " + p.price);
        }

        // Switch statements (bad)
        switch(type){
            case "Electronics":
System.out.println("Apply electronics rules");
break;
            case "Clothes":
System.out.println("Apply clothes rules");
break;
            case "Books":
System.out.println("Apply books rules");
break;
        }

        switch(customerType){
```

```
}
```

Another

```
class OnlineShop {
    List<Product> products;

    void printProducts() {
        for(Product p: products) p.printInfo();
    }

    int calculateTotal() {
        int total = 0;
        for(Product p: products) total +=
p.getPrice();
        return total;
    }

    void checkout(boolean specialDay,
boolean vipCustomer) {
        int discount = 0;
        if (specialDay) discount += 10;
        if (vipCustomer) discount += 5;
        System.out.println("Discount applied:
" + discount);

        printProducts();
        System.out.println("Total: " +
(calculateTotal() - discount));
    }
}

abstract class ProductCategory {
    abstract void applyCategoryRules();
}

class ElectronicsCategory extends
ProductCategory {
    void applyCategoryRules() {
```

```
            case "Regular":
System.out.println("Regular customer");
break;
            case "VIP": System.out.println("VIP
customer"); break;
        }

        int total = 0;
        for(Product p: products) total += p.price;
        System.out.println("Total: " + (total -
tempDiscount));
    }
}

class Product {
    String name;
    int price;
}
```

```
System.out.println("Apply electronics
rules"); }
}

class ClothesCategory extends
ProductCategory {
    void applyCategoryRules() {
System.out.println("Apply clothes rules");
}
}

class BooksCategory extends
ProductCategory {
    void applyCategoryRules() {
System.out.println("Apply books rules"); }
}

abstract class CustomerType {
    abstract void applyCustomerRules();
}

class RegularCustomer extends
CustomerType {
    void applyCustomerRules() {
System.out.println("Regular customer"); }
}

class VIPCustomer extends
CustomerType {
    void applyCustomerRules() {
System.out.println("VIP customer"); }
}

class Product {
    private String name;
    private int price;

    Product(String name, int price) {
this.name = name; this.price = price; }

    void printInfo() {
        System.out.println("Product: " +
name);
        System.out.println("Price: " + price);
    }
```

	int getPrice() { return price; } }

Exam

```
public class Employee {
    protected String name;
```

```
    protected String department;

    public void logWorkingHours(int hours) {
        System.out.println(name + " logged " +
hours + " hours.");
    }

    public void applyForLeave(int days) {
        System.out.println(name + " applied for " +
days + " days of leave.");
    }
}

public class Contractor extends Employee {
    @Override
    public void logWorkingHours(int hours) {
        System.out.println("Contractor " + name +
" logged " + hours + " hours.");
    }

    @Override
    public void applyForLeave(int days) {
```

```
        throw new
UnsupportedOperationException("Contractors
cannot apply");
    }
}
```

After

```
interface WorkLogger {
    void logWorkingHours(int hours);
}

class Employee implements WorkLogger {
    protected String name;
    protected String department;

    @Override
    public void logWorkingHours(int hours) {
        System.out.println(name + " logged " +
hours + " hours.");
    }

    public void applyForLeave(int days) {
```

```
        System.out.println(name + " applied for " +
days + " days of leave.");
    }
}

class Contractor implements WorkLogger {
    protected String name;

    @Override
    public void logWorkingHours(int hours) {
        System.out.println("Contractor " + name +
" logged " + hours + " hours.");
    }
}
```

Previous lab:

```
import java.util.*;

// Observer
interface User {
    void update(String content);
}
```

```
// ConcreteObserver
class EditorUser implements User {
    private String name;

    public EditorUser(String name) { this.name =
name; }

    @Override
    public void update(String content) {
        System.out.println(name + " received
update: " + content);
    }
}

// Subject
class Document {
    private List<User> users = new ArrayList<>();
    private String content;

    public void attach(User user) {
users.add(user); }
```

```
    public void detach(User user) {
users.remove(user); }

    public void editContent(String newContent) {
        this.content = newContent;
        notifyAllUsers();
    }

    private void notifyAllUsers() {
        for (User u : users) {
            u.update(content);
        }
    }
}

// Client
public class Main {
    public static void main(String[] args) {
        Document doc = new Document();

        User user1 = new EditorUser("Alice");
        User user2 = new EditorUser("Bob");
```

```
        User user3 = new EditorUser("Charlie");

        doc.attach(user1);
        doc.attach(user2);
        doc.attach(user3);

        doc.editContent("User Alice edited the
introduction section.");
    }
}
```

Another:

```
import java.util.*;

// Component
interface MenuComponent {
    void display();
}

// Leaf
class Dish implements MenuComponent {
    private String name;
```

```
    public Dish(String name) { this.name = name;
}

    public void display() {
        System.out.println("Dish: " + name);
    }
}

// Composite
class MenuCategory implements
MenuComponent {
    private String name;
    private List<MenuComponent> items = new
ArrayList<>();

    public MenuCategory(String name) {
this.name = name; }

    public void add(MenuComponent item) {
items.add(item); }

    public void display() {
```

```
        System.out.println("Category: " + name);
        for (MenuComponent item : items) {
            item.display();
        }
    }
}

// Client
public class Main {
    public static void main(String[] args) {
        Dish pasta = new Dish("Pasta");
        Dish pizza = new Dish("Pizza");
        Dish brownie = new Dish("Chocolate
Brownie");

        MenuCategory desserts = new
MenuCategory("Desserts");
        desserts.add(brownie);

        MenuCategory mains = new
MenuCategory("Main Courses");
        mains.add(pasta);
```

```
        mains.add(pizza);

        MenuCategory menu = new
MenuCategory("Full Menu");
        menu.add(mains);
        menu.add(desserts);

        menu.display();
    }
}
```